

DataExpert.io Community Academy — Week 4 Grader Feedback

Student

Name: Ceasar Jackson

Final Grade

B

Grader Feedback

Ceasar, thanks for the clear, well-commented submission. You've done solid work on state tracking and window-function problems, and your use of `GROUPING SETS` is on the right track. Below is detailed feedback per requirement, plus what's missing and how to fix it quickly.

High-level summary

What's solid

- Query 1 (state changes) is thoughtfully designed and produces the right categories.
- Query 2 (``GROUPING SETS``) is correct and efficient for multi-dimensional aggregates.
- Query 6 and 7 (window functions) are correctly implemented with sensible ordering and frames.

What's missing / needs changes

- Explicit answers (top-1 results) were not provided for Queries 3, 4, and 5.
- The ``GROUPING SETS`` query produces the aggregates needed, but the assignment requires concrete outputs for those questions.
- Minor efficiency and edge-case improvements are available in Queries 1, 6, and 7.

Detailed review by requirement

1) Query 1 — Player state change tracking

File: 01_player_state_change_tracking.sql

Correctness

- Correctly identifies New, Continued Playing, Retired, Returned from Retirement, and Stayed Retired.
- Filtering out “Not Yet in League” keeps the output relevant.

Edge cases

Seasons past a player’s final active season are labeled Stayed Retired across the dataset’s maximum season. This matches the definitions, but can inflate “retired-time” if the dataset spans far beyond a player’s career. Evaluation could instead be constrained to each player’s minimum/maximum active-season window.

Efficiency

The solution generates a full grid (all players × all seasons) and then filters. The grid could be tightened to each player’s minimum/maximum seasons to reduce work.

Clarity

Very readable; the flag CTE makes the CASE logic easy to verify.

2) Query 2 — GROUPING SETS across player-team, player-season, and team

File: 02_grouping_sets_game_details.sql

Correctness

- `GROUPING SETS` and `GROUPING()` flags are used properly to distinguish aggregation levels.
- Distinct counting in team wins avoids duplicate-count inflation from the player grain.

What’s missing for Queries 3–5

The submission stops at returning the entire aggregated set. The assignment also asks for:

1. Query 3 — player who scored the most points for a single team.
2. Query 4 — player who scored the most points in a single season.
3. Query 5 — team with the most total wins.

Small final SELECT statements should produce those exact answers.

Important: Query 5 must order by `team_wins`, not `total_points`.

Minimal fixes

```
-- Query 3
SELECT player_name, team_abbreviation, total_points
FROM grouped_metrics
WHERE aggregation_level = 'player_team'
ORDER BY total_points DESC NULLS LAST
LIMIT 1;
```

```
-- Query 4
SELECT player_name, season, total_points
FROM grouped_metrics
WHERE aggregation_level = 'player_season'
ORDER BY total_points DESC NULLS LAST
LIMIT 1;
```

```
-- Query 5
SELECT team_abbreviation, team_wins
FROM grouped_metrics
WHERE aggregation_level = 'team'
ORDER BY team_wins DESC NULLS LAST
LIMIT 1;
```

Efficiency

Precomputing team-game results is a good approach. Win counting can be simplified by using that CTE consistently and avoiding unnecessary distinct operations at player grain.

Clarity

Clear labeling via `aggregation_level` helps substantially.

3) Query 3 — Player who scored the most points for a single team

Not submitted as a discrete query. Add a top-1 SELECT filtering `aggregation_level = 'player_team'`.

4) Query 4 — Player who scored the most points in a single season

Not submitted as a discrete query. Add a top-1 SELECT filtering `aggregation_level = 'player_season'`.

5) Query 5 — Team with the most total wins

Not submitted as a discrete query. Add a top-1 SELECT filtering `aggregation_level = 'team'` and order by `team_wins DESC`.

6) Query 6 — Most games a team has won in a 90-game stretch

File: 03_window_functions_game_details.sql, Question 1

Correctness

- `SUM(is_win) OVER (PARTITION BY team_id ORDER BY game_date_est, game_id ROWS BETWEEN 89 PRECEDING AND CURRENT ROW)` is correct.
- Filtering to complete 90-game windows is correct.

Efficiency

Building team games by `DISTINCT`-ing `game_details` works but is heavier than necessary. Team-game rows can instead be created directly from games: one row for the home team and one for the visitor.

Ties and output

- `RANK()` could return all teams tied for the maximum instead of `LIMIT 1`.
- The starting game/date could also be included by deriving the 90th preceding game.

7) Query 7 — Longest LeBron James streak over 10 points

File: 03_window_functions_game_details.sql, Question 2

Correctness

The cumulative streak-group / gaps-and-islands technique is correct and clean. Sorting by game date and game ID gives a deterministic sequence.

Edge cases

If the dataset includes DNP rows with zero/null points, those rows will break a streak. If the requirement means “games he played,” participation could be filtered using minutes or another reliable played indicator.

Output

The query returns start/end dates and streak length, fully answering the question.

Submission completeness

The grader assessed the original submission as implementing **4 of 7 required explicit outputs**:

- Query 1 — implemented.
- Query 2 — implemented.
- Query 3 — missing as a discrete output.
- Query 4 — missing as a discrete output.
- Query 5 — missing as a discrete output.
- Query 6 — implemented.
- Query 7 — implemented.

Queries 3–5 should be supplied as explicit queries even though Query 2 already computes their underlying aggregates.

Environment / assumptions noted by grader

- The SQL uses PostgreSQL-oriented features such as `generate_series`` and `GROUPING()`.
- If `game_details`` encodes DNPs differently, the participation column should be confirmed before refining Query 7.
- A separate reporting layer does not replace the homework requirement to include the specific top-1 queries in the submission.

Actionable next steps from grader

1. Add a top-1 query for player with most points for a single team.
2. Add a top-1 query for player with most points in a single season.
3. Add a top-1 query for team with most total wins.
4. Optionally optimize Query 6 to derive team-game rows directly from `games``.
5. Optionally refine Query 7 to exclude DNPs if that matches the intended scoring-streak definition.

Remediation status

This feedback has already been addressed in the Week 4 remediation work:

- Seven explicit SQL outputs were created.
- Query 5 orders by `team_wins DESC``.
- The remediation validator passed **18 / 18** checks.
- The original B-grade baseline was preserved.
- A fresh seven-query resubmission ZIP was validated.